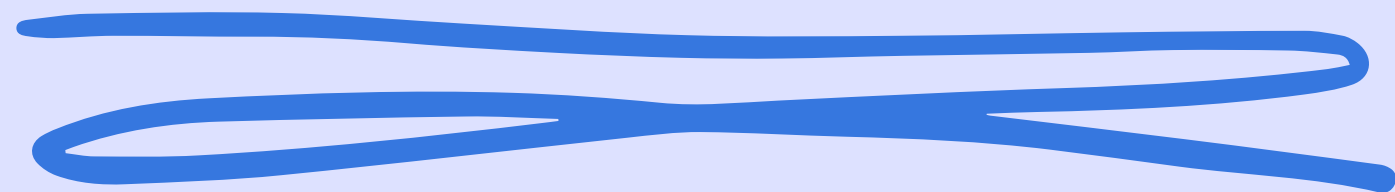




Ruggine

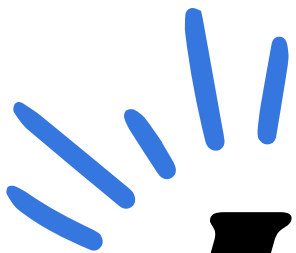


Giuliano Adriani

Edoardo Borlina

Federico Ferrari

Pasquale Papalia



Indice

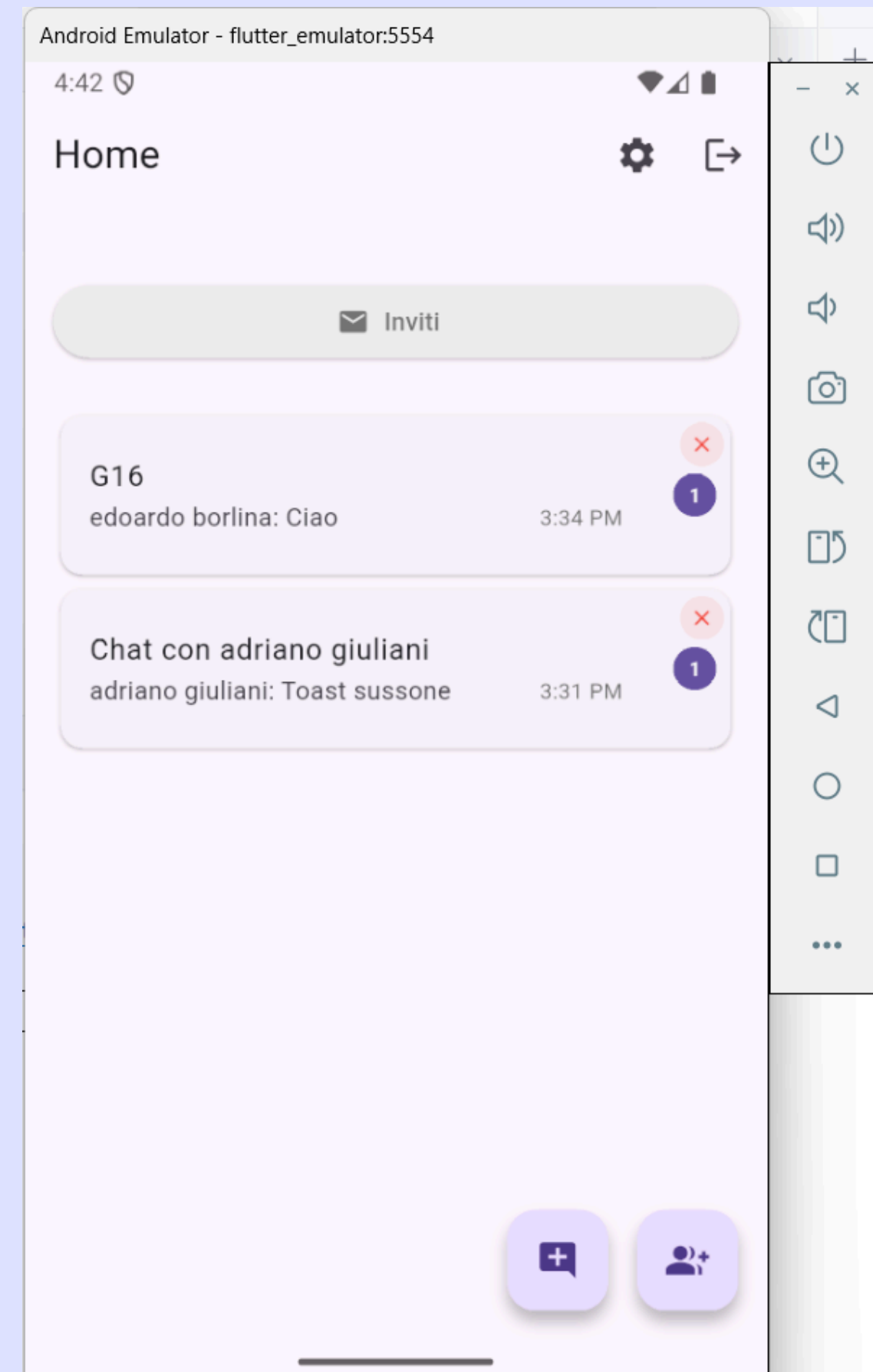


- Introduzione
- Backend
 - Tecnologie utilizzate
 - Struttura del progetto
 - Struttura del database
 - Monitoraggio CPU
- Frontend
 - Tecnologie utilizzate
 - Architettura client
 - Persistenza
- Safety
- Sviluppi futuri

Introduzione

Ruggine è un'applicazione per inviare messaggi di testo tra utenti in chat private oppure in gruppi di più utenti.

Ciascun utente, dopo aver fatto login, può creare un nuovo gruppo e invitare altri utenti. L'applicazione utilizza un'architettura **client-server**, composta da un backend sviluppato con **Rust** e da un frontend sviluppato con **Flutter**, in modo da poter essere utilizzata su diversi sistemi operativi: Windows, Android, MacOS...





Problemi e soluzioni

Applicazione multiplatforma

Abbiamo utilizzato Flutter per garantire una compilazione del client multiplatforma

Affidabilità e performance

Abbiamo utilizzato Rust per garantire un uso efficiente delle risorse,

Notifiche sincrone

Abbiamo utilizzato i web sockets per garantire notifiche istantanee verso i client che ricevono nuovi messaggi o inviti



Backend

Actix web

Web framework rust più popolare



Diesel ORM

Object-Relational Mapping per gestire le entità del database

Sqlite

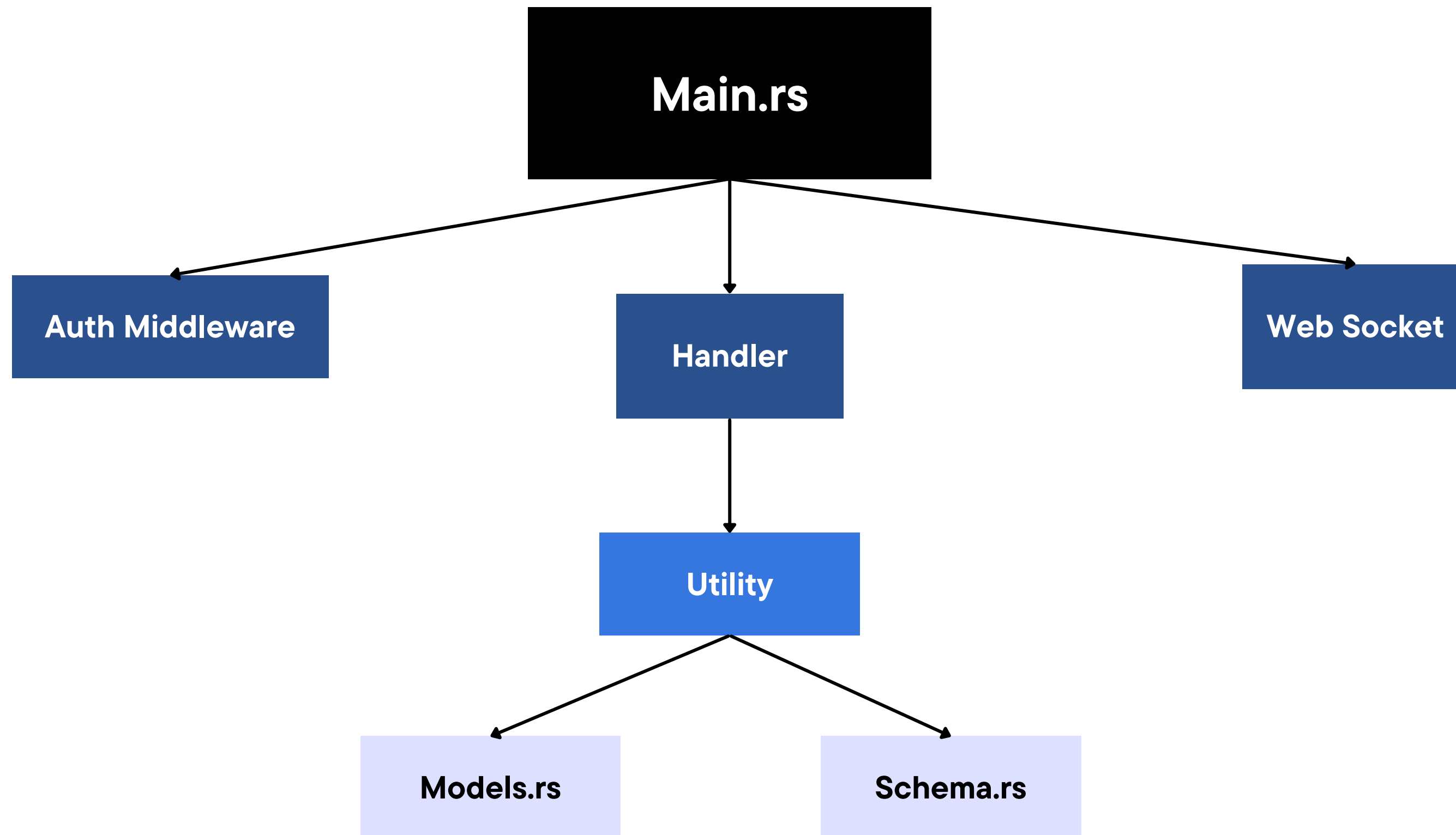
Database locale salvato su file

Sysinfo

Per ottenere informazioni sul consumo di risorse del sistema



Struttura del backend



Handler

Viene definito un handler per ciascun endpoint.

Il compito di un handler è di gestire le chiamate ad uno specifico endpoint, controllando le informazioni contenute nell'header e nel body.

Restituire una risposta con un eventuale body in formato JSON

Web Socket

Permette una connessione bidirezionale e persistente tra client e server.

Gestisce l'aggiunta e la rimozione delle connessioni attive in una mappa condivisa (ClientSockets), associando ogni utente al proprio socket.

Una funzione viene chiamata quando un client si connette all'endpoint WebSocket, avviando la sessione per l'utente autenticato.

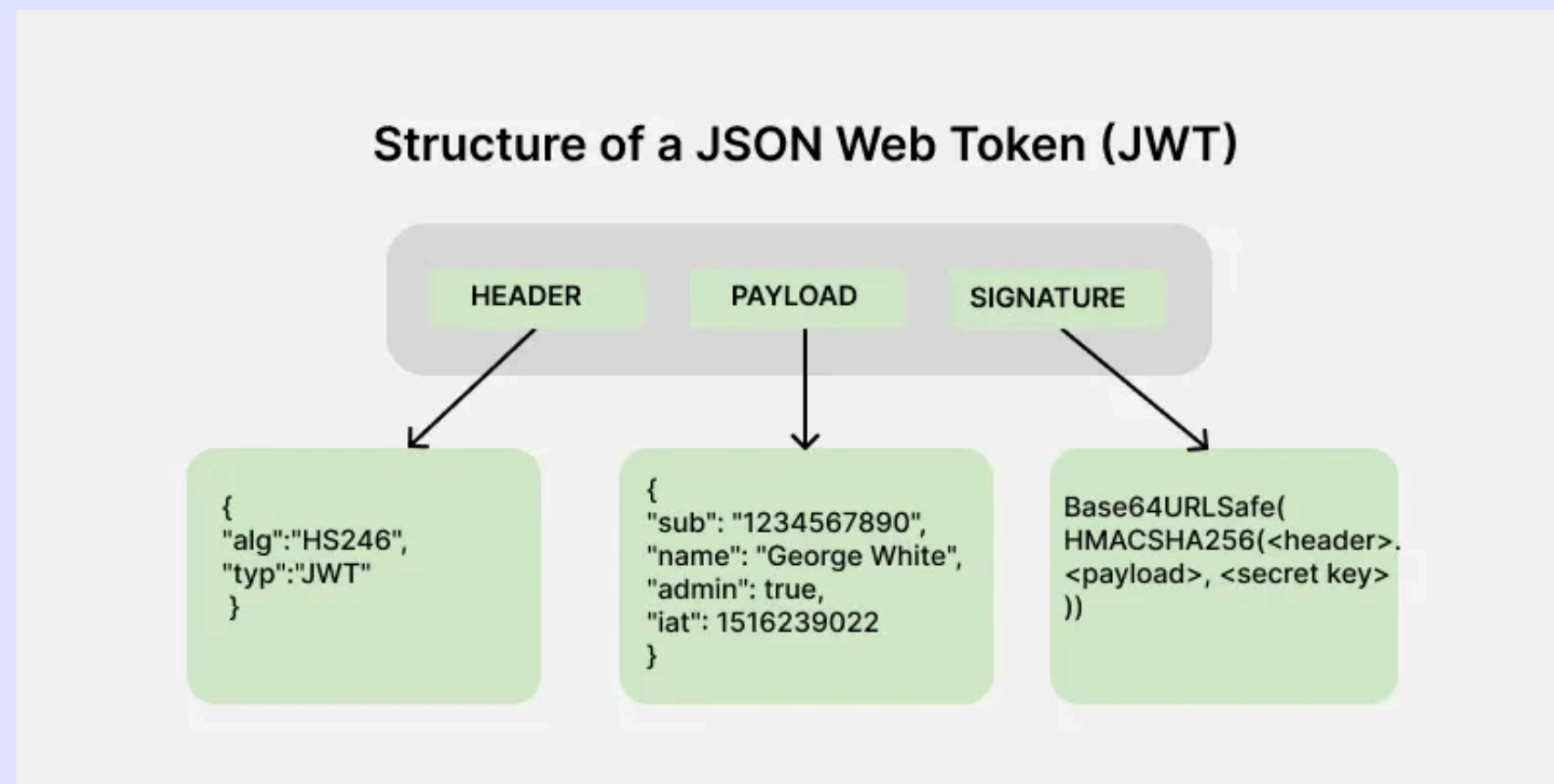
Auth Middleware

L'auth middleware serve a proteggere gli endpoint dell'applicazione verificando che ogni richiesta HTTP abbia un token JWT valido nell'header Authorization.

Se il token è valido, estrae i dati dell'utente (claims) e li rende disponibili agli handler successivi; se manca o non è valido, blocca la richiesta restituendo un errore di autorizzazione.

In questo modo l'utente dal lato frontend può salvarsi il Token e inserirlo negli header della chiamata successiva.

Per gestire la sicurezza ciascun token è firmato con una chiave segreta salvata nelle variabili d'ambiente. Se un utente prova a modificare i dati contenuti nel token questo viene invalidato



Utility

I file dentro la cartella utility contengono la business logic del progetto, implementando le funzioni per gestire con il database attraverso i modelli.

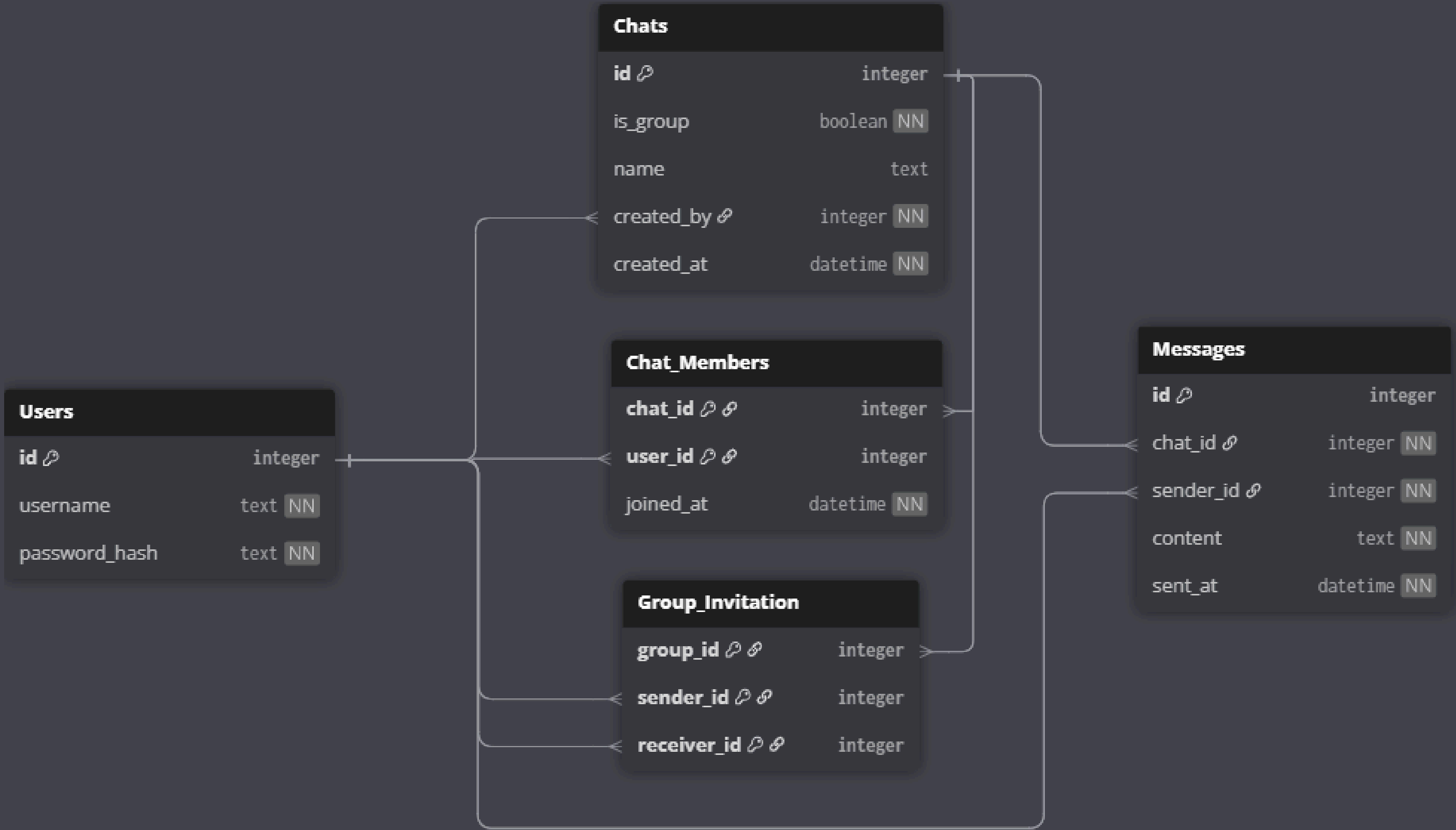
Models

Definisce le strutture Rust che rappresentano le tabelle del database

Schema

Viene Generato da Diesel e descrive la struttura delle tabelle del database

Struttura del Database



Monitoraggio della CPU

Una funzione asincrona registra periodicamente l'utilizzo della CPU del processo server. Ogni 120 secondi, raccoglie il dato di utilizzo CPU, lo formatta con timestamp e PID, e lo scrive in un file di log. Il monitoraggio avviene tramite la libreria sysinfo, che permette di ottenere informazioni sul processo corrente

1	Timestamp: 2025-07-12 13:50:43.497438400 UTC, PID: 3292, CPU Usage: 4.02%
2	Timestamp: 2025-07-12 13:52:43.521841 UTC, PID: 3292, CPU Usage: 0.81%
3	Timestamp: 2025-07-12 13:54:14, PID: 5248, CPU Usage: 2.17%
4	Timestamp: 2025-07-12 13:56:22, PID: 4552, CPU Usage: 0.93%
5	Timestamp: 2025-07-12 13:56:27, PID: 4552, CPU Usage: 0.00%
6	Timestamp: 2025-07-12 13:56:32, PID: 4552, CPU Usage: 0.00%
7	Timestamp: 2025-07-12 13:56:37, PID: 4552, CPU Usage: 0.00%
8	Timestamp: 2025-07-12 13:56:42, PID: 4552, CPU Usage: 2.49%
9	Timestamp: 2025-07-12 13:56:47, PID: 4552, CPU Usage: 0.00%
10	Timestamp: 2025-07-12 13:56:52, PID: 4552, CPU Usage: 0.94%
11	Timestamp: 2025-07-12 13:56:57, PID: 4552, CPU Usage: 1.87%
12	Timestamp: 2025-07-12 13:57:02, PID: 4552, CPU Usage: 0.00%
13	Timestamp: 2025-07-12 13:57:07, PID: 4552, CPU Usage: 0.00%
14	Timestamp: 2025-07-12 13:57:12, PID: 4552, CPU Usage: 0.00%
15	Timestamp: 2025-07-12 13:57:17, PID: 4552, CPU Usage: 0.00%
16	Timestamp: 2025-07-12 13:57:22, PID: 4552, CPU Usage: 0.62%
17	Timestamp: 2025-07-12 13:57:27, PID: 4552, CPU Usage: 0.62%
18	Timestamp: 2025-07-12 13:57:32, PID: 4552, CPU Usage: 4.38%
19	Timestamp: 2025-07-12 13:57:37, PID: 4552, CPU Usage: 0.00%

Architettura Client

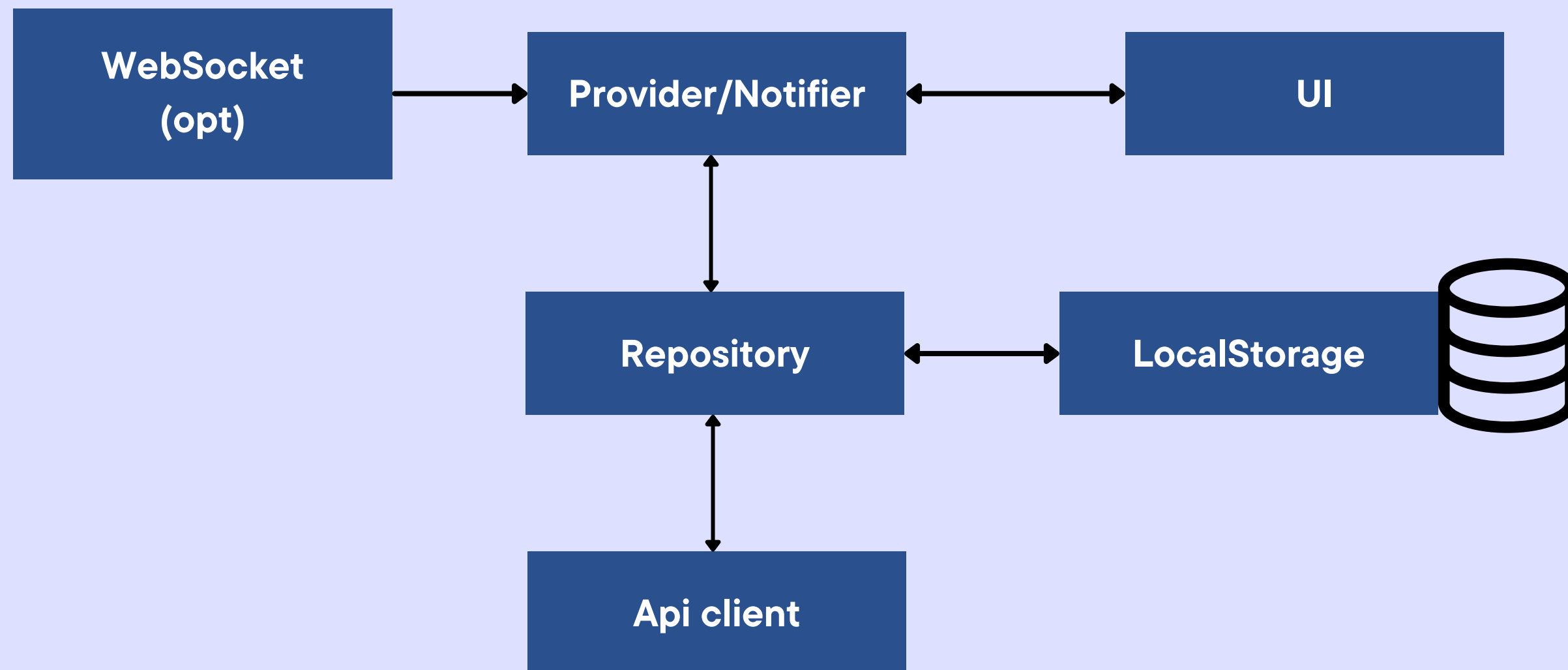
Per il client è stato scelto flutter per via di:

- Sviluppo one-for-all, un solo codice su tutte le piattaforme (nel nostro caso windows e android)
- Rapidità di integrazione data l'ampia varietà di librerie presenti
- Predisposizione ad approccio RESTful utilizzando DIO library
- Reattività UI e aggiornamento stati tramite Riverpod



Architettura Client

L'approccio utilizzato si basa sull'utilizzo di Riverpod per la gestione dello stato (Provider), organizzando Chat, Inviti e Messaggi (oltre ad autenticazione, websocket e tema) con un approccio a livelli



Architettura Client - Persistenza

Per quanto riguarda il salvataggio dei dati in locale, sono stati utilizzati sia SecureStorage che LocalStorage rispettivamente per token JWT e Chat/Messaggi, richiedendo così al server solo le informazioni mancanti.



Live delivery

Grazie all'implementazione dei websocket all'avvio della sessione autenticata dell'utente, se esso riceve un messaggio o un invito ed il websocket è attualmente registrato come connesso, il messaggio/invito è recapitato in diretta e notificato al provider, la logica è identica a quella per messaggi recuperati all'avvio.





Backend Safety

Utilizzando Rust come linguaggio di programmazione per il nostro server abbiamo diverse garanzie:

[Torna all'indice](#)



Memory safety

Previene buffer overflow, memory leaks e data races a compile-time



Ownership system

Gestione automatica della memoria senza garbage collection



Error handling con Result<T,E>:

Gestione esplicita degli errori senza eccezioni impreviste



Lifetime management

Prevenzione di dangling pointers e use-after-free



Sviluppi futuri

[Torna all'indice](#)



- * **Aggiunta nuovi tipi di messaggi**
Aggiungere per esempio contunuti multimediali ed emoticon
- * **Cifratura end to end**
In modo tale che i messaggi non vengano salvati in chiaro sul db
- * **Integrazione servizi esterni**
Implementare altri servzi, come ad esempio notifiche push
- * **Migrazione a DB distribuiti**
Utilizzo di DBMS in grado di gestire db condivisi, come posgreSQL

Grazie per l'attenzione!



[Torna all'indice](#)